시스템 연동 · 장비 제어 · 로봇 SW

김소정

KIM SOJEONG

사용자 화면부터 장비 동작까지, 데이터와 상태 흐름을 연결하는 개발자

ksj9164@gmail.com



대표 성과

9년
C++/MFC 상용 SW 개발·운영
그룹사 통합 메신저 구조·DB 설계, Teams 연동, WinDbg 장애 분석

8종
다중 로봇 교통 시나리오 구축
병목·마주 오는 주행·3대 순환 대기를 시간 예약과 양보로 해소하고 웹·CLI로 검증

20+곳
고객사별 통신·연동 요구 대응
HTTPS 통신, XML→JSON 변환, 보안 SDK 연동을 고객사마다 맞춰 구현

바로 기여

C++ 기반 운영 프로그램 개발, 외부 시스템 연동, DB 구조 설계와 데이터 이전, 사용자 환경 장애 분석에 바로 기여할 수 있습니다.

넓혀 가는 중

ROS 2·Nav2 기반 다중 로봇 경로·예약 제어와, 센서·설비 상태를 화면과 동기화하는 영역으로 역량을 넓혀 가고 있습니다.

경력 · 교육

2026.04 – 2026.11 **대한상공회의소 서울기술교육센터** 교육
AI융합 로봇 SW 개발자 과정 3기 · C/C++, Python, Linux · PLC(MELSEC-Q, GOT HMI, Servo) · STM32 HAL · ROS 2 다중 로봇 물류 프로젝트 진행 중

2017.08 – 2024.08 **큐텐테크놀로지 · 선임연구원** 실무
사내 메신저 Qpost 클라이언트(MFC) 개발·운영 · 그룹사 통합 메신저 구조·DB 설계와 데이터 마이그레이션 · Microsoft Teams 채팅 연동(Graph REST API) · 보안 관리 프로그램 SecurityManager 개발 · Teams 조직도 앱(C# Blazor) 유지보수

2014.11 – 2016.09 **소프트보울 · 사원** 실무
XML/XForms 웹 UI 개발 도구 TrustForm(MFC) 유지보수 · 신한카드, 국립중앙의료원 등 20여 개 고객사 전담 · HTTPS 통신, XML→JSON 변환, 키보드 보안 모듈(nProtect) 연동

2013.12 – 2014.10 **콤텍정보통신 · 앱 개발** 실무
한국공항공사 항공종합통신망(ATN) 데이터베이스 시스템 · Java Swing 검색·관리 터미널, Oracle 기반 CRUD 구현

2010.02 – 2014.02 **호서대학교 정보통신공학과 졸업**
학점 3.72 / 4.5 · 전공 동아리 DOIT(C/C++·AVR 스터디) · C언어 튜터링 진행

ABOUT · SKILLS & EXPERIENCE

화면 안의 소프트웨어에서, 장비를 움직이는 소프트웨어로

9년 동안 C++/MFC로 업무 시스템과 사용자를 연결하는 클라이언트를 만들었습니다. 여러 시스템 사이의 데이터 흐름을 파악하고 문제의 근본 원인을 찾는 일에 가장 몰입했지만, 그 결과는 늘 화면 위에서 끝났습니다. 같은 방식으로 실제 장비를 움직이고 싶어 PLC, STM32, ROS 2를 차례로 익혔고, 지금은 다중 로봇 관제 시스템에서 경로 탐색과 교통 제어를 맡고 있습니다.

기술 역량 · 개발 경험

01 다중 로봇 경로·교통 제어

Python · ROS 2 · Nav2 · Gazebo · Linux (Ubuntu)

- Space-Time A* 경로 탐색과 노드·통로 시간 예약표로 로봇 간 충돌을 사전에 차단
- 순환 대기를 탐지해 측면 노드로 양보시킨 뒤 원래 목적지로 복귀
- 예약·교착·주행 허가 정책을 실물 연동이 가능한 독립 서비스로 분리

근거 · **P1 다중 로봇 FMS**

02 시스템 연동·데이터 흐름 설계

C++ · SQL · SQLite · REST API (Microsoft Graph)

- 분리된 메신저를 통합 구조·DB로 재설계하고 로컬 DB 마이그레이션 구현
- Graph REST API로 사내 메신저와 Teams 간 메시지 조회·전송·동기화
- 계열사별 조직도를 기준으로 사용자를 관리하는 DB 테이블 설계

근거 · **P4 통합 메신저**

03 장비 제어·임베디드 구현

C · STM32 HAL · STM32CubeIDE · MELSEC PLC

- STM32 HAL로 가속도 센서(I2C)와 TFT LCD(SPI)를 연동해 상태 표시
- PLC 래더로 적재·비움 수동 시퀀스를 만들고 HMI 화면과 연동
- GOT HMI 4개 화면을 구성하고 램프·메시지를 PLC 상태와 동기화

근거 · **P2 STM32 RC카** · **P3 AutoStore PLC**

04 장애 분석·안정화

Windows · Visual Studio · WinDbg

- WinDbg 크래시 덤프·호출 스택으로 사용자 환경 전용 비정상 종료 해결
- LCD·SD카드 문제를 소프트웨어 → 전압 → 배선 순으로 좁혀 해결
- 교통 시나리오 8종을 웹·CLI에서 실행해 변경 후 회귀 확인

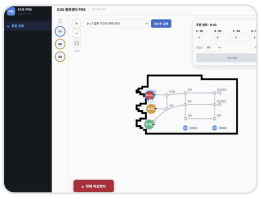
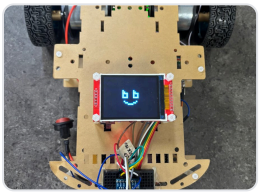
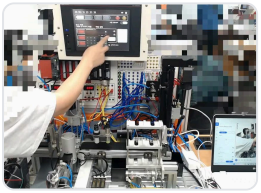
근거 · **P4** · **P2** · **P1**

협업 도구 Git · GitHub · Jira · Confluence

PROJECTS AT A GLANCE

교육 프로젝트 3건과 실무 프로젝트 1건

최근 프로젝트부터 정리했습니다. 각 프로젝트에서 맡은 범위만 적고, 팀 전체의 결과와 구분했습니다.

	프로젝트	기간 · 인원	담당 · 결과
	P1 · p.04–05 **다중 로봇 물류 자동화 FMS** AMR 3대와 로봇팔 4대를 주문 단위로 운영하는 관제 시스템	2026.09 – 진행 중 7인 진행 중	경로 탐색, 노드·통로 시간 예약, 교착 해소, 주행 허가 로직 분리 **교통 시나리오 8종 검증 체계 구축**
	P2 · p.06–07 **STM32 무선 RC카** 조이스틱 무선 조종과 기울기에 반응하는 LCD 표시	2026.07 – 08 4인	가속도 센서(I2C)와 TFT LCD(SPI) 연동, 표시 안정화 **기울기 3단계 상태 전환, 변화 영역만 다시 그림**
	P3 · p.08–09 **AutoStore 가공·적재 설비** 가공 조건에 따라 창고 위치를 정하는 PLC 설비 제어	2026.06 2인	수동 시퀀스(지정 적재·선택 비움), HMI 화면 구성·PLC 연동 **PLC 동작과 HMI 표시 시점 불일치 해결**
C++ · MFC · SQLite Messenger A·B·C → Qpost	P4 · p.10–11 **그룹사 통합 메신저 Qpost** 분리 운영되던 사내 메신저를 하나의 구조로 통합	2017.08 – 2024.08 큐텐테크놀로지 실무	통합 구조·DB 설계, 로컬 DB 마이그레이션, Teams 연동, 크래시 분석 **사용자 환경 전용 비정상 종료 원인 규명·수정**

읽는 방법 프로젝트마다 **개요 → 내가 맡은 일 → 문제 · 해결 · 결과** 순서입니다. 수치는 코드, 커밋, 발표 자료에서 확인한 값만 사용했습니다.

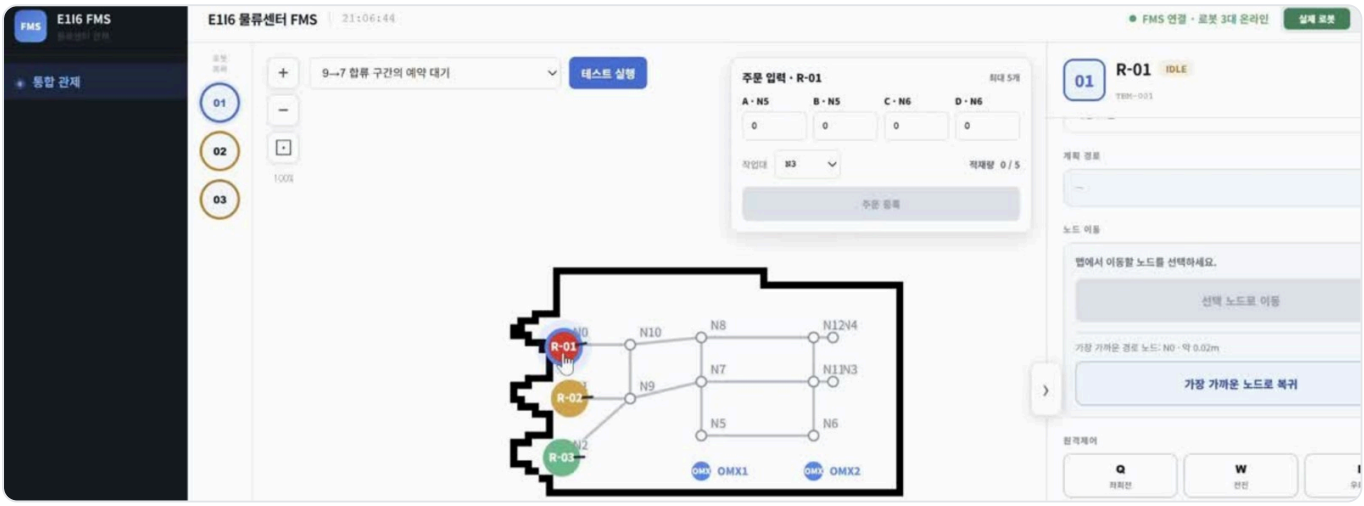
P1 MULTI-ROBOT FLEET MANAGEMENT

다중 로봇 물류 자동화 FMS

주문이 들어오면 AMR을 배차하고, 피킹 스테이션에서 로봇팔이 물품을 싣고, 패킹 스테이션까지 운반하는 흐름을 관제합니다. 저는 FMS 파트에서 **여러 AMR이 서로 막히지 않도록 경로와 통행 시간을 조율하는 부분**을 맡았습니다.

기간	인원	역할	규모
2026.09 – 진행 중	7인 (FMS · AMR · OMX)	FMS 경로·교통 제어	AMR 3대 · 로봇팔 4대

Python · FastAPI · ROS 2 Jazzy · Nav2 · Zenoh · MQTT · WebSocket · React · pytest



FMS 웹 관제 화면. 노드 그래프 지도 위 로봇 3대의 위치와 상태가 실시간으로 갱신되고, 상단 콤보박스에서 교통 시나리오를 골라 실행합니다.

시스템 구성 · 담당 영역



경로 탐색

A*로 시작해 예약 상태를 반영하는 **Space-Time A***로 확장. 기다릴지 돌아갈지를 도착 시각으로 비교합니다.

시간 예약표

노드와 통로를 자원으로 보고, 로봇마다 **언제부터 언제까지** 쓰는지 기록합니다. 시간 구간이 겹치면 확정하지 않습니다.

교통 관리 분리

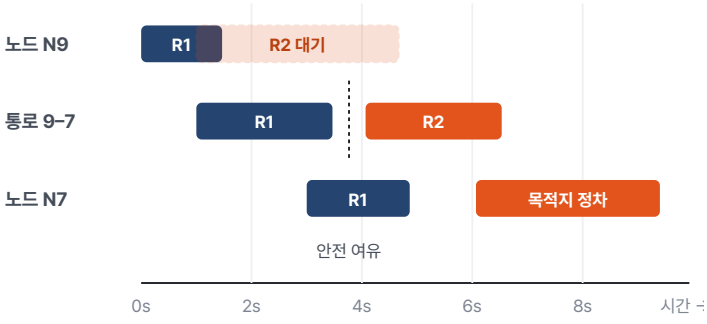
시뮬레이터 안에 섞여 있던 예약·교착·허가 정책을 **TrafficManager(459줄)**로 옮겨 실물 실행기와 연결할 수 있게 했습니다.

협업 사례 배차 시 최적 경로 계산을 Nav 파트와 FMS 파트 중 어디서 맡을지 의견이 갈려 같은 논의가 반복됐습니다. 합의가 필요한 쟁점을 다시 정의하고 현업 시스템의 구조와 역할 분담 사례를 확인해 판단 기준을 만들었고, 그 기준으로 두 파트의 담당 범위를 다시 나눠 합의했습니다.

P1 ROUTE RESERVATION & DEADLOCK

"어느 길로"에 "언제"를 더해 로봇끼리 막히지 않게

시간 예약표 예시 · 9→7 합류 구간



R1이 통로를 쓰는 시간과 겹치지 않도록 R2는 N9에서 기다린 뒤 진입합니다. 방향이 달라도 같은 물리 통로는 하나의 자원 키 (`"edge", 작은 ID, 큰 ID`)로 비교합니다. 목적지 정차도 예약해 다른 로봇의 진입을 막습니다.

실행 시점의 주행 허가

예약이 있어도 실제 위치가 계획과 다를 수 있어, 구간마다 출발 직전에 다시 확인합니다. 허가 판단은 상태와 예약표를 바꾸지 않습니다.

allowed	요청·구간 예약, 출발 시각, 실제 점유, 예상 도착 시각 검사를 모두 통과
waiting	계획된 출발 시각 전. 예약은 그대로 둠
replan	예약 누락, 다른 로봇 점유, 시간표 초과. **정지가 확인된 뒤에만** 예약 해제

문제 · 해결 · 결과

통로 하나를 두고 마주침

PROBLEM

처음에는 경로 전체를 한 로봇이 독점하도록 예약해, 대기 시간이 길고 마주 오는 통로에서 멈췄습니다.

SOLUTION

구간별 시간 창으로 쪼개 예약하고, 같은 자원에서 시간이 겹칠 때만 충돌로 판단했습니다.

RESULT

병목·교차점에서 로봇이 순서대로 통과합니다.

3대가 서로를 기다림

PROBLEM

로봇들이 서로의 다음 노드를 점유하면 아무도 움직이지 못했습니다.

SOLUTION

대기 관계에서 순환을 찾아 한 대를 빈 측면 노드로 양보시키고, 대피 뒤에도 진행 가능한지 복사본 상태로 먼저 검증했습니다.

RESULT

양보한 로봇은 원래 목적지로 다시 계획해 복귀합니다.

재계획 후 남은 이전 경로

PROBLEM

브랜치 병합 후 재계획에 실패하면 이전 경로가 화면과 상태에 남았습니다.

SOLUTION

초기화 누락 지점을 찾아, 실패하면 기존 route를 비우거나 직전 상태를 보존하도록 분기했습니다.

RESULT

목적지 변경이 실패해도 상태가 엇갈리지 않습니다.

검증 · 교통 시나리오 8종 (웹 · CLI 단일 실행)

`bottleneck` 9→7 합류 구간 예약 대기	`head_on` 두 로봇 자리 교환과 양보	`cycle` 세 로봇 순환 대기 해소	`cycle_reverse` 역순 요청의 순환 대기
`central_head_on` 중앙 통로 마주침·우회	`junction` 교차점 세 방향 순차 통과	`swap_crossing` 자리 교환 중 제3 로봇 통과	`central_cycle` 중앙 순환, 양보 후 복귀

시뮬레이션은 시작 위치를 복원해 30초 안에, 실물은 배치 위치를 0.15m 허용 오차로 확인한 뒤 180초 안에 끝나야 통과입니다. 실물 시작 위치가 틀리면 명령을 보내기 전에 거부 사유를 보여 줍니다.

진행 현황 실물 TurtleBot3 3대와 통합 검증 중입니다. 점유·예약 해제와 주행 흐름을 점검하고 있으며, 같은 조건에서 LLM 경로 판단과 비교하는 실험을 팀과 함께 설계하고 있습니다.

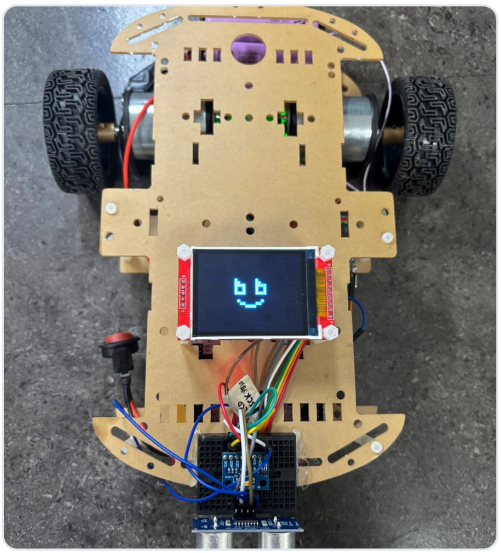
P2 EMBEDDED FIRMWARE

STM32 무선 RC카
기울기에 반응하는 LCD 표시

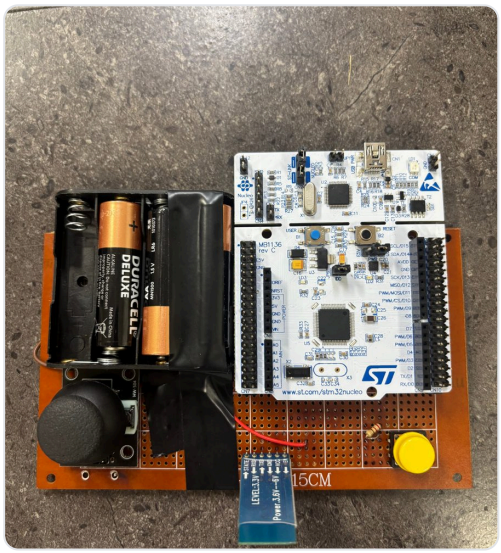
STM32F411RE 보드 두 대를 블루투스 UART로 연결해 조이스틱으로 조종하는 RC카입니다. 저는 차체의 기울기를 읽어 LCD 캐릭터의 위치와 표정을 바꾸는 **센서-화면 연동**을 맡았습니다.

기간	인원	역할	보드
2026.07 – 08	4인	센서·디스플레이 연동	STM32F411RE × 2

C · STM32 HAL · ADXL345 (I2C) · ILI9341 TFT (SPI) · UART · Bluetooth · TIM · PWM



차체 · 상단 LCD에 기울기 캐릭터



조종기 · 조이스틱 + 블루투스



주행 시연

데이터 흐름 · 담당 영역

조이스틱 ADC·DMA — 10ms 샘플 · 이동평균 10개 → **UART 패킷 9바이트** — 100ms 송신 · 0xAA 0×55 시작 → **링버퍼 파싱** — 500ms 무수신 시 정지 → **PI 모터 제어** — 엔코더 RPM · 10ms

팀 구현 (조종기 → 차체 구동)

담당 영역

ADXL345 읽기 — I2C · 50ms · Z축 오프셋 → **좌표 정규화** — 데드존 + 3제곱 이징 곡선 → **상태 판정** — LEFT · CENTER · RIGHT → **ILI9341 갱신** — SPI · 변화 영역만

팀 결과

조이스틱 입력 → 패킷 송수신 → 좌우 모터 PI 제어를 10ms 주기로 안정적으로 운용했고, 초음파 10cm 장애물 정지와 500ms fail-safe를 함께 동작시켰습니다. 계획했던 카메라(OV7670) 영상 저장은 기간 안에 완성하지 못해 최종 시스템에서 제외했습니다.

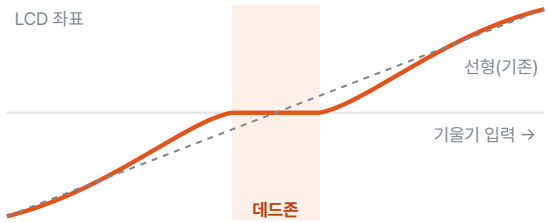
P2 STABILIZATION & TROUBLESHOOTING

흔들림 없는 화면, 신호와 배선까지 확인하는 검증

표시 안정화 1

미세 진동에 화면이 떨리지 않게

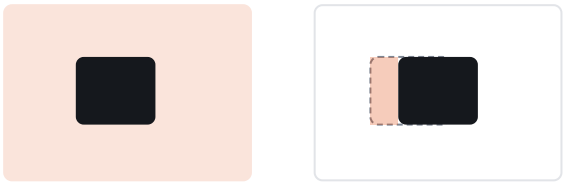
가속도 원시값을 그대로 좌표로 쓰면 주행 중 작은 진동에도 캐릭터가 흔들렸습니다. 중앙 부근에 **데드존**을 두고, 기울기가 커질수록 빠르게 움직이는 **3제곱 이징 곡선**으로 바꿔 작은 흔들림은 무시하고 의도한 기울기만 반영했습니다.



표시 안정화 2

바뀐 부분만 다시 그리기

매 주기 화면 전체를 지우고 다시 그리면 깜빡임이 보이고 SPI 전송량이 늘었습니다. 캐릭터의 **이전 위치와 겹치지 않는 영역만 배경색으로 지우고** 새 위치를 그리는 부분 갱신 방식으로 바꿔 깜빡임과 불필요한 전송을 줄였습니다.



전체 갱신 · 매 주기 화면 전체

부분 갱신 · 비워진 띠만 지움

트러블슈팅

LCD ID 값이 예상과 다르게 읽힘

증상	RDDID(0×04) 명령으로 컨트롤러 ID를 읽으면 기대값(0x93, 0×41)과 달랐습니다.
원인	MISO 배선이 연결되지 않아 플로팅 상태의 핀 값이 우연히 읽혔습니다.
해결	FillScreen 색 채우기로 초기화 성공을 별도로 눈으로 확인하고 배선을 점검했습니다.
배운 점	레지스터 읽기는 배선 상태에 좌우되므로 화면 출력 검증을 함께 둡니다.

LCD 모듈의 SD카드 인식 실패

증상	LCD 모듈에 달린 SD카드가 인식되지 않고 읽기에 실패했습니다.
원인	코드 → 전압(3.3V→5V) 순으로 가설을 지운 끝에, DC/RS 핀 자리에 SD_CS가 꽂힌 배선 오류를 찾았습니다.
해결	핀맵을 다시 대조해 배선을 바로잡고 정상 인식을 확인했습니다.
배운 점	가설을 하나씩 지우며 원인 범위를 좁히는 순서를 지킵니다.

펌웨어 주기 설계 · TIM5 1MS 소프트 타이머

`10 ms`	조이스틱 → 목표 RPM, 장애물 확인, 엔코더 RPM 읽기, PI 연산, PWM 출력
`50 ms`	UART 패킷 파싱과 fail-safe 판정, **가속도 읽기와 LCD 캐릭터 갱신 (담당)**
`100 ms`	초음파 트리거, 조이스틱 패킷 송신

팀 저장소 · github.com/heeyeon5877-tech/Lcd_RC_Car_With_STM32

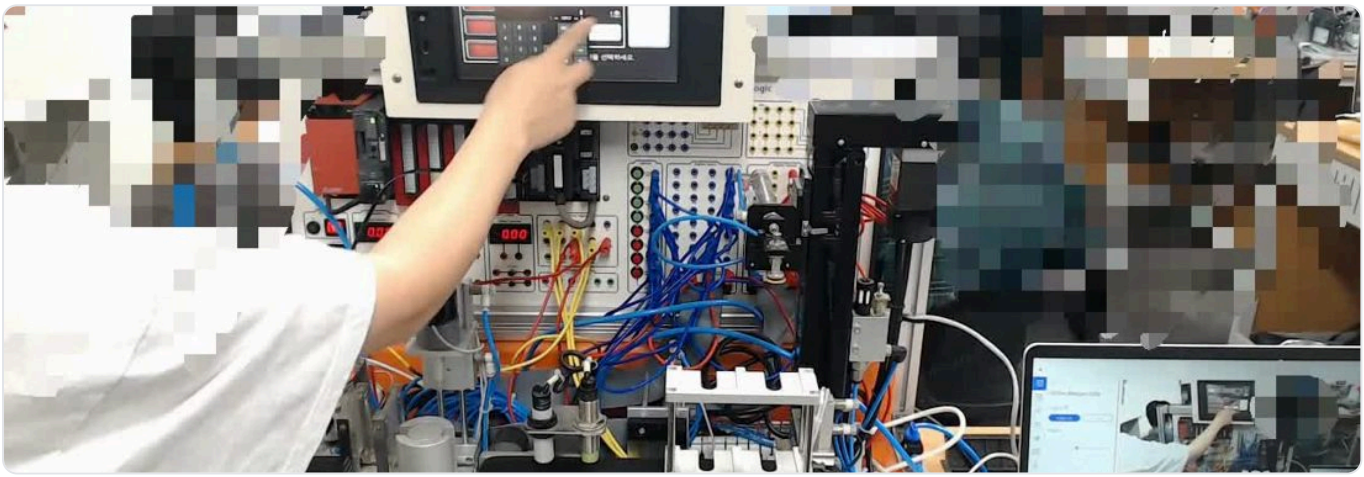
P3 PLC & HMI

AutoStore
가공 조건으로 적재 위치를 정하는 설비

금속·비금속 소재를 드릴로 가공하고, 가공 시간과 횟수에 따라 창고 6칸 중 적재 위치를 정하는 설비입니다. 저는 **수동 시퀀스와 HMI 화면 구성·PLC 연동**을, 팀원은 자동 시퀀스를 맡았습니다.

기간	인원	역할	PLC
2026.06	2인	수동 시퀀스 · HMI	MELSEC-Q (Q03UDV)

Ladder · GX Works2 · GOT HMI · Servo · 센서 · 드릴 · 실린더



실제 설비에서 HMI로 수동 적재를 실행하는 모습 (인물·주변 모자이크)

적재 위치 결정 규칙



가공 시간으로 열을, 가공 횟수로 층을 정합니다. 칸별 소재 수는 D101–D106에 저장합니다.

담당 기능

- **지정 적재(수동)** · 가공 시간·횟수를 입력하면 드릴 동작 후 위치를 정해 적재하고, 끝나면 시퀀스를 초기화합니다.
- **선택 비움(수동)** · 적재된 칸을 골라 해당 위치의 소재만 비웁니다.
- **사전 조건 확인** · 적재 전에는 칸이 비었는지, 비움 전에는 소재가 있는지 확인하고 아니면 하단 메시지로 알립니다.
- **HMI 4개 화면** · 메인, 서보, 자동, 수동 화면을 만들고 버튼·램프·메시지를 PLC 디바이스에 연결했습니다.

P3 HMI SCREENS & STATE SYNC

설비가 움직인 순간, 화면도 같은 상태를 보여 주도록

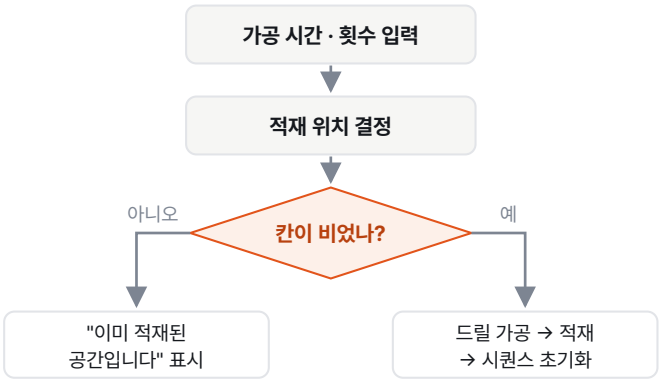


수동 제어 화면 · 창고 램프와 비우기 버튼, 가공 조건 입력



서보 제어 화면 · 현재 위치·속도, JOG, 원점 복귀

사전 조건 확인 흐름 (지정 적재)



메시지 화면





조건이 맞지 않으면 동작하지 않고 이유를 화면 하단에 알립니다.

PLC 동작과 HMI 램프·메시지의 표시 시점이 어긋남

PROBLEM	설비는 적재를 끝냈는데 HMI 램프가 늦게 켜지거나 이전 메시지가 남았습니다.
CAUSE	PLC 내부 비트와 HMI 표시용 디바이스가 분리되어, 상태가 바뀌는 시점과 화면 값이 갱신되는 시점이 달랐습니다.
SOLUTION	동작 완료 시점에 표시용 디바이스를 함께 갱신하도록 내부 상태 연동 로직을 고쳤습니다.
RESULT	적재·비움이 끝나는 순간 램프와 메시지가 설비 상태와 같은 값을 보여 줍니다.

P4 WORK EXPERIENCE · C++ / MFC

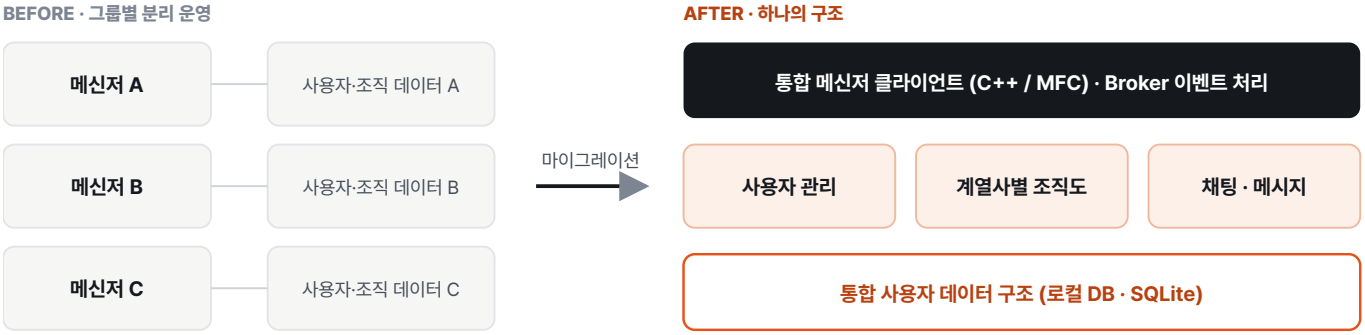
그룹사 통합 메신저 Qpost
구조·DB 설계와 데이터 이전

그룹사가 늘면서 업무 목적과 사용자에 따라 따로 운영되던 메신저를 하나로 재구축했습니다. 기능을 단순히 합치지 않고, 사용자 데이터가 어디서 생겨 어떤 경로로 화면에 닿는지부터 분석했습니다.

회사	기간	직급	환경
큐텐테크놀로지	2017.08 – 2024.08	선임연구원	Windows 클라이언트

C++ · MFC · SQLite · Broker 라이브러리 · GitHub 코드 리뷰

기존 구조 → 통합 구조



담당 범위

- **구조 설계** · 기존 메신저의 구조와 데이터 흐름을 분석해 통합 메신저 구조를 설계
- **DB 설계** · 계열사별 조직도를 기준으로 사용자를 관리하는 테이블 구성
- **마이그레이션** · 기존 사용자의 로컬 DB를 통합 구조로 옮기는 전환 로직 개발
- **이벤트 처리** · 메신저 내부 이벤트를 Broker 라이브러리로 처리
- **핵심 화면** · 로그인, 조직도, 채팅 목록, 모바일 포워딩, 설정

그 밖의 7년간 개발

- 그룹사 친구 추가·삭제, 스티커, 오픈 채팅
- 신규 그룹사 전용 메신저와 조직도 추가
- 출퇴근 체크인·체크아웃, 모바일 수신 전용 채팅창
- 메신저 전체 디자인 리뉴얼
- **SecurityManager** · PC의 Hotfix 내역과 설치 프로그램을 서버와 동기화하고, 패치·삭제 알림을 받는 구조 설계
- Teams 조직도 앱(C# Blazor) 기능 추가

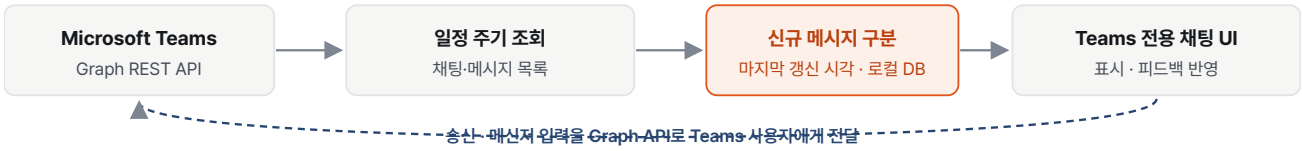
로봇 SW로 이어지는 점 흩어진 데이터를 하나의 상태 구조로 모으는 일은 FMS에서 로봇·주문·예약 상태를 한곳에서 관리하는 일과 같은 문제입니다.

P4 INTEGRATION & CRASH ANALYSIS

외부 시스템을 연결하고, 재현되지 않는 장애를 추적하다

A · MICROSOFT TEAMS 채팅 연동

사내 메신저에서 Teams 사용자와 대화하기



API를 일정 주기로 조회하되 **마지막 업데이트 시각을 로컬 DB에 저장해 비교**하는 방식으로 새 메시지만 골라, 같은 메시지가 두 번 뜨거나 빠지는 경우를 줄였습니다. 전용 채팅 UI는 사용자 피드백을 받아 여러 차례 개선했습니다.

B · 사용자 환경에서만 발생한 비정상 종료

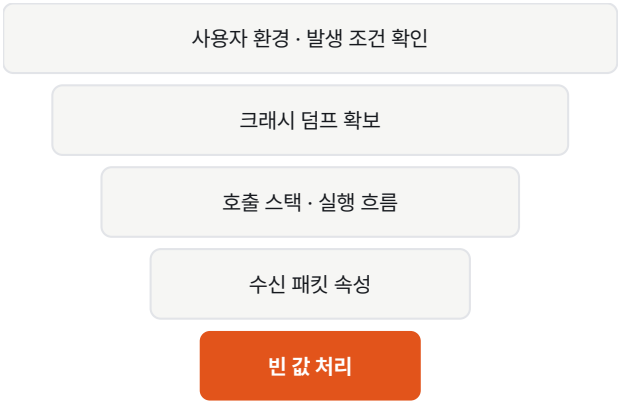
WinDbg로 크래시 덤프를 따라가 원인 확인

증상 특정 사용자 PC에서만 메신저가 종료되고, 개발 환경에서는 재현되지 않았습니다.

추적 발생 당시 사용 조건을 확인하고, 크래시 덤프의 호출 스택으로 실행 흐름을 거슬러 올라갔습니다.

원인 모바일 앱으로 대화하던 상대가 PC 메신저로 넘어와 대화를 이어갈 때, 수신 패킷의 특정 속성이 비어 있어 메모리 참조 오류가 났습니다.

결과 빈 값을 처리하도록 수정해 종료를 막았고, 이후 새 기능마다 빈 값 같은 예외 조건을 먼저 점검합니다.



C · 이전 회사 소프트보울 (2014–2016) · TRUSTFORM

MFC로 만든 XML/XForms 웹 UI 도구를 유지보수하며 **신한카드·국립중앙의료원 등 20여 개 고객사**를 전담했습니다. 고객사마다 다른 요구에 맞춰 WinInet HTTPS 통신, XML→JSON 변환 API, HTTP 오류 시 에러 페이지 전환을 구현했고, 키보드 보안 모듈(nProtect NOS)을 화면 생성·포커스 진입·이탈 시점에 맞춰 연동했습니다.

CLOSING

현장의 문제를 소프트웨어로 해결하는 로봇·장비 SW 개발자로 성장하겠습니다

담당 장비의 동작 순서와 기존 구조부터 익히고, 맡은 기능은 예외 상황까지 검증해 끝까지 책임지겠습니다.

EMAIL	ksj9164@gmail.com
GITHUB	github.com/sojeong-grotto
WEB	sojeong-grotto.github.io
FMS	github.com/E1I6-Logistics/Logistics_FMS
STM32	github.com/heeyeon5877-tech/Lcd_RC_Car_With_STM32

감사합니다.